

Lab Sheet 4: Software System Modelling

Objectives

By completing today's workshop, you will:

- Practice translating system requirements into formal **UML Models**.
 - Map the actors and system boundaries using a **Use Case Diagram**.
 - Model the business logic and decisions using an **Activity Diagram**.
 - Define the OOP architecture and data structures using a **Class Diagram**.
 - Trace the chronological execution of API commands using a **Sequence Diagram**.
-

Introduction: Modelling in Practice

The goal of this workshop is to practice abstracting your software system into visual models. Before you write the core logic for your Assessment 1 Robot Management System, you must fully map out its architecture.

To draw these diagrams, we will use **Mermaid.js**. Mermaid allows you to write markdown-style text that automatically renders into diagrams directly on GitHub.

*Note: While Mermaid is the tool, the focus of today is the **modelling process**. You must think critically about the relationships, logic, and data structures in your system.*

To test your diagrams, use the **Mermaid Live Editor**: <https://mermaid.live> before pasting the code into your GitHub README.md or docs/ folder.

Task 1: The Use Case Diagram

Goal: Define the external perspective. Who uses the Ground Control Station and what can they do?

The Concept: We need to visually map the Role-Based Access Control (RBAC) requirements. You have multiple actors (Commanders, Viewers, Auditors) and multiple features.

Action:

1. Open the Mermaid Live Editor.
2. Draft a Use Case Diagram mapping your actors to their permissions. Here is some starter code to visualize the syntax:

```
flowchart LR
    %% Define Actors
    C[Commander]
    V[Viewer]
```

```
%% Define Use Cases
Move((Move Robot))
Status((View Status))

%% Connect Actors to Use Cases
C --> Move
C --> Status
V --> Status
```

3. **Extend the Model:** Expand this diagram to include the **Auditor** actor and the **Reset Robot** use case. Ensure lines are only drawn to the actors who have permission for those specific actions.
4. Copy your final code into your GitHub repository inside a ‘‘**mermaid** code block, for example in a new file Markdown file in a **docs/** folder. See guidance in the resources section below for how to render Mermaid diagrams on GitHub.

Task 2: The Activity Diagram

Goal: Model the behavioral perspective and business logic of a single process.

The Concept: An Activity Diagram shows the flow of control, including decision points (diamonds). Let’s model the logic the backend must execute when a user attempts to send a move command.

Action:

1. Create a new diagram to map the “Move Robot” authorization process.
2. Use this base structure:

```
stateDiagram-v2
    [*] --> ReceiveRequest
    ReceiveRequest --> CheckRole

    state check_role <<choice>>
    CheckRole --> check_role

    check_role --> RejectCommand : Role is Viewer
    check_role --> SendToRobot : Role is Commander

    RejectCommand --> [*]
    SendToRobot --> [*]
```

3. **Extend the Model:** In Assessment 1, the connection to the robot might drop. Add a new decision node after **SendToRobot** to check if the robot API is responsive. If yes, go to **LogSuccess**. If no, go to **LogError**.

Task 3: The Class Diagram (OOP Data Models)

Goal: Model the structural perspective of your backend data.

The Concept: You need to design the Object-Oriented architecture for your backend. This involves defining Classes, Attributes (data), and Operations (methods).

Action:

1. Draft a Class Diagram mapping out the core entities of your application.
2. Use the 'classDiagram' syntax:

```
classDiagram
    class User {
        -String username
        -String passwordHash
        -String role
        +login() bool
        +getRole() String
    }

    class RobotController {
        -String apiEndpoint
        +getStatus() JSON
        +moveRobot(int x, int y) bool
    }

    User "1" --> "1" RobotController : Uses
```

3. **Extend the Model:** Add a MissionLog class to represent the audit trail requirement. What attributes should it store? Add a composition/aggregation line showing how the RobotController interacts with the MissionLog.

Task 4: The Sequence Diagram

Goal: Trace the interaction perspective over time.

The Concept: Sequence diagrams show how objects communicate chronologically. We will trace a command from the human user all the way to the Docker simulation.

Action:

1. Draft the timeline of a "Move" command.

```
sequenceDiagram
    actor C as Commander
    participant UI as Web Dashboard
    participant API as Python/Node Backend
    participant Sim as Virtual Robot (Docker)

    C->>UI: Enter X, Y and click 'Move'
    UI->>API: POST /api/command {x: 5, y: 10}

    activate API
    API->>Sim: POST /api/move {x: 5, y: 10}
    Sim-->>API: 200 OK
```

```
API-->>UI: 200 OK
deactivate API
```

2. **Extend the Model:** Add the missing steps! Before the API sends the command to the **Sim**, it should verify the user's token. After the **Sim** responds, the API should write to a **Database** participant to log the mission. Update the diagram to reflect these critical architectural steps.

Task 5 (Distinction Stretch): Component Diagram

Goal: Model the high-level physical architecture of your system.

The Concept: A Component Diagram requires us to look at the system macroscopically. How do the frontend, backend, database, and virtual robot connect over the network? To learn more about Component Diagrams, you can refer to the IBM UML Documentation or the Lucidchart Component Diagram Guide.

Action:

1. Use Mermaid to create a high-level Component/Architecture diagram showing your deployment strategy.
2. Group components using subgraphs (e.g., wrap your Web UI and Backend API inside a 'subgraph Ground Control Station', and place the 'Virtual Robot Container' outside of it).
3. Add this final architectural blueprint to the root 'README.md' of your Assessment repository.

Further Resources

- **Mermaid Live Editor:** <https://mermaid.live/> - *Test all diagrams here before committing!*
- **GitHub Docs: Creating Mermaid Diagrams:** <https://docs.github.com/en/.../creating-diagrams>
- **UML Class Diagram Reference:** IBM UML Documentation
- **UML Component Diagram Reference:** IBM UML Documentation
- **Mermaid Syntax Docs:** <https://mermaid.js.org/intro/>